Technical Screening — Round 1 (Take-Home)

About This Round

This is an asynchronous take-home assessment. You have 48 **hours** from the time you receive this to submit your solution.

AI usage is explicitly allowed and encouraged. You may use ChatGPT, Claude, GitHub Copilot, or any other AI tool. We are evaluating *how* you use AI, not *whether* you use it. Treat this the way you would approach a real engineering task at work.

Problem Statement

PDF Generation Microservice for an ERP Application

We need to build a PDF generation microservice for an ERP application that handles both individual and bulk document generation.

Core Requirements

- The service receives HTTP requests with JSON payloads containing data for purchase orders and invoices from the primary service.
- Two types of requests must be supported:
  - **Single document generation:** One PDF at a time
  - **Bulk generation:** Multiple documents (up to 100 PDFs) in one request
- The service should generate PDFs using Puppeteer based on HTML templates.
- Users should be able to download generated PDFs (single file or zip file for bulk).
- The service should handle concurrent requests efficiently.
- Expected load: ~1000 single requests/min during peak times, and 5–10 bulk requests/min.

Performance & Resource Profile

- Each PDF generation takes 2–3 seconds and consumes ~400MB memory temporarily.
- Individual PDFs are typically 100KB–1MB in size.
- Bulk requests should provide progress updates to users.
- Generated PDFs may need to be archived for compliance purposes.

- Users expect:
  - Single downloads to start within 5 seconds
  - Bulk downloads to provide status updates and complete within reasonable time

---

Scenario Constraints

Read these carefully — they affect your design.

1. **Existing Infrastructure:** The primary ERP service is a **monolithic Django application** running on a single EC2 instance (r6i.large, 2 vCPUs, 16GB RAM). Your PDF service must integrate with this existing setup without requiring the monolith to be re-architected. The Django app currently uses PostgreSQL (RDS db.r6i.large) and Redis for caching.
2. **Variable Document Complexity:** Some purchase orders contain **dynamic tables** with variable row counts (5 to 500+ line items). PDFs with 500+ line items frequently cause Puppeteer to crash mid-render with OOM errors. Your design must handle this gracefully.
3. **Unreliable Client Connectivity:** Your customers are small and medium businesses. Some of them operate on **unreliable internet connections** — bulk downloads of 50+ PDFs frequently fail midway through the download. Your design must account for this.
4. **Data Consistency:** A customer recently reported that their **GST invoice PDF showed stale data** because the ERP data was updated between the time they triggered bulk generation and the time the 87th PDF was rendered (roughly 3 minutes later). How does your design handle data consistency for bulk operations?
5. **Tamper Evidence:** The compliance team requires that generated PDFs must be **tamper-evident** — they need to be able to prove a PDF was not modified after generation. How do you handle this without overcomplicating the architecture?
6. **Budget Constraint:** The total monthly infrastructure cost for this service must stay **under ₹12,500 (~$150 USD)** for a customer generating approximately 30,000 PDFs/month.

What You Need to Submit

1. Architecture Diagram

A high-level design diagram of your solution. This can be hand-drawn (photo), created with draw.io/excalidraw, or any tool you prefer. It should clearly show:

- All services/components and how they communicate
- Data flow for both single and bulk requests
- Storage, queue, and caching layers
- External integrations (if any)

2. APPROACH.md

This is the most important part of your submission. Use the provided template (below) and fill it out thoroughly. We weight this document heavily in our evaluation.

3. Your AI Chat Conversations

Make the chat conversations that you had with AI public and share the links with us.

APPROACH.md Template

Use the following structure for your APPROACH.md. Be honest and thorough — we learn more from your reasoning than from a polished final answer.

```
# Approach Document

## 1. Initial Understanding
> Before using any AI tools or doing research, what was your initial
read of this
> problem? What did you think the hard parts were?

[Your response here]


## 2. Assumptions & Clarifying Questions
> List any assumptions you made. In a real scenario, what questions
would you ask the
> product team or engineering lead before starting?

[Your response here]


## 3. Capacity Planning & Math
> Show your calculations. How did you arrive at your resource
requirements?
> Consider: memory, CPU, storage, network bandwidth, queue depth.

[Your response here]


## 4. Design Decisions (minimum 3)
> For each major decision, document:
> - What alternatives you considered
> - Why you chose the approach you did
> - What tradeoff you're accepting

### Decision 1: [Title]
**Alternatives considered:**
**Chosen approach:**
**Why:**
**Tradeoff accepted:**

### Decision 2: [Title]
...
```

```
### Decision 3: [Title]
...


## 5. AI Usage Log
> Public links of conversations you had with AI.



## 6. Weaknesses & Future Improvements
> What is the weakest part of your design? If you had 2 more days, what
would you
> improve? What would you change if this needed to support 10x the load?

[Your response here]


## 7. One Thing the Problem Statement Didn't Mention
> What's something important for a production deployment that wasn't
called out in the
> requirements? Why does it matter?

[Your response here]


## 8. Cost Estimation
Provide a line-by-line monthly cost breakdown for your proposed
infrastructure.
Show your math. We care more about the reasoning than the exact numbers.
```

Evaluation Criteria

We are evaluating your submission across four dimensions. Here is exactly what we look for:

Dimension	What We're Looking For
Technical Depth	Capacity math, awareness of failure modes, understanding of infrastructure costs, data consistency reasoning
Problem Decomposition	How you broke down the problem, sequencing of your approach, identification of constraints not explicitly stated
Practical Engineering Sense	Right-sizing the solution for the scale, budget-awareness, considering the end-

	user experience (unreliable internet, etc.)
AI Collaboration	Effective prompting, critical evaluation of AI output, knowing when to override AI suggestions, using AI for exploration vs. just generation

What We Don't Care About

- Pixel-perfect diagrams
- Using the "right" buzzwords
- A perfect solution (there isn't one — we want to see your tradeoffs)

What Will Stand Out

- Honest documentation of dead ends and mistakes
- Calculations that show you actually did the math
- Catching where AI gave you a generic or wrong answer and correcting it
- Identifying real-world production concerns beyond what was asked

---

Submission Instructions

1. Create a **private GitHub repository** with your submission.
2. Include:
   - `APPROACH.md` (using the template above)
   - `architecture-diagram.png` (or `.pdf`, `.svg`)
   - Any additional notes or code snippets (optional)
3. Add `sun337` and `rss-holmes` as collaborators on the repo.
4. Reply to the email you received with the repository link.

Deadline: 48 hours from the time this was sent to you.

Good luck. We're looking forward to seeing how you think.